Today

Welcome to your next lesson: Simple invariants: what must always stay true.

I'm preparing your personalized lesson based on what you've learned, what's coming next, and how you learn best.

Step by step, you’re moving forward.

Ron, let’s start with something concrete. 🎯

Imagine a game character with a health value.

At 100: perfectly healthy
At 0: knocked out
Below 0 or above 100: makes no sense in the game

So the real-world rule is:

Health should never be less than 0 and never more than 100.

This is the kind of rule we’ll turn into an invariant in code.

Quick check for you:

If some buggy code sets health = -30, what feels more reasonable for the game to do?

Treat -30 as 0 (minimum)
Allow -30 and continue

Just answer in words for now.

cap it at 0 so change it to 0 if < 0

Exactly 👍 — that “cap it at 0” idea is the heart of what we want.

So our informal rule is:

Health is never below 0 and never above 100.

In code, that rule is what we call an invariant:

Invariant: a rule about an object’s state that must always stay true, not just once.

For our character, one invariant could be:

“health is always between 0 and 100.”

Quick check before we touch code:

If we apply this invariant every time health changes, which of these should be fixed (changed) and which are already okay?

health = 50
health = -10
health = 120

You can just reply like: 1 ok, 2 fix, 3 fix (or similar).

1 ok 2 fix 3 fix

Perfect ✅

So:

50 → already valid
−10 → should be corrected (up to 0)
120 → should be corrected (down to 100)

That “correct when out of bounds” is how we enforce the invariant in code.

Next, I want to show this as a simple visual “health bar” over time, like:

120 → 100 → 80 → −5 → 0

You’ll see which steps are invalid and how we fix them.

Ready for a quick visual of this health timeline?

Yes, show me

Take a look at the health timeline below 👇

Follow it from left to right and notice:

Which points are red (break the rule: not between 0 and 100)
How each red point has an arrow down/up to a green corrected value
In your own words: what is this picture showing about how we keep the health invariant true over time?

Health Invariant Timeline

it shows that values from 0 to 100 stay as is but values < 0

are remapped to 0 and values > 100 are remapped to 100

Exactly. 🎯 Nicely put.

What you just described is our invariant in action:

Keep health between 0 and 100 by:

leaving values in [0, 100] alone
turning < 0 into 0
turning > 100 into 100

Now let’s turn that into a small Python class.

Step 1: we’ll make a Player with a health attribute and one method that sets health while enforcing this rule.

Can you open the code editor and write this, step by step:

Create a class Player with a constructor that takes health and stores it in self.health without any checks yet.
Add a method set_health(self, new_health) that:
if new_health < 0, stores 0
elif new_health > 100, stores 100
else stores new_health
At the bottom, create one Player, call set_health a few times with values like 120, 80, -5, and print player.health after each call.

Then hit Submit code, and I’ll review it. 💻

Python

```
class Player:
    def __init__(self):
        self.health = 100
    def set_health(self, new_health):
        if new_health < 0:
            self.health = 0
        elif new_health > 100:
            self.health = 100
```

```
        else:
            self.health = new_health

ron = Player()
ron.set_health(55)
print(ron.health)
ron.set_health(160)
print(ron.health)
ron.set_health(130)
print(ron.health)
Nice work, Ron 💪 Your code does exactly what we described:


Start at 100
Set to 55 → stays 55 (already in range)
Set to 160 → capped at 100
Set to 130 → still 100 (already capped)

So your invariant is:



health is always between 0 and 100, no matter what values
set_health gets.


One more step: I want to show how this helps in a multi-step
situation.


Imagine this sequence:


health = 50
Take 70 damage → health would try to go to -20
Our invariant should correct this to 0 so later code never
sees a negative health.

We'll add a take_damage(amount) method that subtracts from
health but still respects the 0–100 rule.


Can you update your class in the editor to:


Keep your existing set_health.
Add take_damage(self, amount) that:
```

computes new_health = self.health - amount
then calls self.set_health(new_health) to enforce the invariant.
Create a player at 50 health, call take_damage(70), and print health.

Run and Submit code, and we'll check that health never goes below 0.

Python

```
class Player:
    def __init__(self):
        self.health = 100
    def set_health(self, new_health):
        if new_health < 0:
            self.health = 0
        elif new_health > 100:
            self.health = 100
        else:
            self.health = new_health

    def take_damage(self, amount):
        new_health = self.health - amount
        self.set_health(new_health)


ron = Player()
ron.set_health(50)
ron.take_damage(70)
print(ron.health)
ron.set_health(160)
print(ron.health)
ron.set_health(130)
print(ron.health)
```

This is excellent, Ron 🎯

Step by step, your code is now guaranteeing the invariant:

Start at 100
set_health(50) → 50 (in range)
take_damage(70) → tries to go to −20, but set_health corrects it to 0
set_health(160) → corrected to 100
set_health(130) → stays 100

So at every point in time, ron.health is between 0 and 100. That’s exactly invariant behavior.

Now I want to check that the idea is clear in your head, not just in code.

Think in words, not code:

We have this multi-step sequence:

Health starts at 30
Heal by 50 (so it wants to go to 80)
Heal by 40 (so it wants to go to 120, but we enforce the same invariant: 0–100)

After step 3, what final health value should we end up with if we keep the invariant?

Just answer with the number.

100

What do you think? Pick the option that best matches the idea of “a rule that must always stay true.” 👇

Your game character’s health must always stay between 0 and 100. Which statement is the invariant?

A.
Health increases after every method call.

B.
Health is always between 0 and 100, inclusive.

C.
Health starts at 100 and never changes.

D.
Health is stored in a class.

🔥 Nailed it, Ron.

You picked:

Health is always between 0 and 100, inclusive.

That is the invariant because:

It's a rule about the state (health)
It must stay true all the time: after constructor, after damage, after healing, after any method call
Your code (set_health and take_damage) is built to protect exactly that rule

The other options talk about behavior ("increases after every method call", "never changes") or where health is stored ("in a class"), but they don't express a "must-always-be-true" rule about valid values.

You've:

Defined an invariant in words
Enforced it in Python with checks and corrections
Seen how it prevents weird behavior in multi-step updates

You've basically acted as the invariant guardian for Player.health 👊

Are you ready to wrap up this lesson and move on to the next one, or do you want one more small invariant example (maybe with something like a bank balance or thermostat) before we finish?

I'm ready to move on

Nice work today, Ron 🎯

Quick recap in plain language:

You defined an invariant: health must always stay between 0 and 100.
You enforced it in code with set_health using simple checks and corrections (capping low values to 0, high values to 100).
You used take_damage to show that even across multiple steps, health never went invalid — no weird negatives, no impossible over-100 values.
You saw how this makes later code more predictable and safer.

You're thinking like someone who's guarding the rules of the system, not just moving numbers around. That's real OOP thinking. 💡

When you're ready, you can jump into the next lesson from the Learn tab — and I'll be right here to keep going step by step with you.

```
class Player:
    def __init__(self):
        self.health = 100
    def set_health(self, new_health):
        if new_health < 0:
            self.health = 0
        elif new_health > 100:
            self.health = 100
        else:
            self.health = new_health

    def take_damage(self, amount):
        new_health = self.health - amount
        self.set_health(new_health)


ron = Player()
ron.set_health(50)
ron.take_damage(70)
print(ron.health)
ron.set_health(160)
print(ron.health)
ron.set_health(130)
print(ron.health)
```